Lecture Slides 2
Object Oriented Programming

Data Encapsulation and Member Functions

AGENDA

The agenda of today's lecture is:

1. Private data members
2. Getter and setter functions (accessors and mutators)
3. Constructors
4. const Member Functions

These features enable **data encapsulation**, **data protection**, and **controlled access** to class data.

1. Private Data Members

In C++, the keyword `private` restricts direct access to class members (variables or functions). Only functions defined inside the class (or friends) can access them.

1.1. Private Data Member Example

```
class Student {
private:
   int rollNo; string name;
public:
   void setData(int r, string n) {
      rollNo = r;
      name = n;
   }
   void display() {
      cout << "Roll No: " <<
      rollNo << ", Name: " << name
};
```

```
int main() {
   Student s;
   //not possible
   //s.rollNo = 101;

   s.setData(101, "Ali");
   s.display();
```

2. GETTERS AND SETTERS (ACCESSORS AND MUTATORS)

Private data ensures protection, but we often need controlled access. To achieve this, we use **getter** and **setter** functions.

2.1. Getter Setters Example

```
class BankAccount {
private: double balance;
public:
  void setBalance(double b) {
     if (b >= 0) balance = b;
     else cout << "Invalid...
    }
  double getBalance() const {
     return balance;
  }
};
```

```
int main() {
  BankAccount acc;
  acc.setBalance(500);
  cout << "Balance: "
   << acc.getBalance()
```

3. Constructors

A constructor is a special member function:

- a non-parameterized constructor exists by default in every class
- automatically called when an object is created
- has the same name as the class
- has no return type, not even void
- we can't call a constructor other than object creation

It is used to initialize object data members.

3.1 Constructor Overloading

A constructor can be overloaded:

- we can write one constructor to overwrite the default constructor

- we may write multiple constructors with different:
  * parameter count
  * parameter type

- Compiler matches the parameters and bind the closest match

3.1. Constructor Example

```
class Rectangle {
private:
   int length;
   int width;
public:
   Rectangle(int l, int w) {
      length = l;
      width = w;
   }
   Rectangle(int s) {
      length = width = s;
   }
int area() const {
   return length*width;
      }
};
int main() {
   Rectangle r1(5, 3);
   Rectangle r2(6);
   cout << r1.area()...
   cout << r2.area()
}
```

4. Destructor

- used to destroy objects and free resources when they go out of scope
- has the same name as the class, preceded by a tilde (~).
- takes no arguments and has no return type
- called automatically when an object’s lifetime ends
- only one destructor is allowed per class (cannot be overloaded)
- in reverse order of object creation
- a default destructor exist in each class

3.1. Constructor Example

```
class A {
private:
   int x;
public:
   A(int x1) { x = x1;}
   ~A() {
      cout << x << " is end ";
   }
};
```

```
void f1(){
     A a(10);
}
int main() {
   A a1(3), a2(4);
   f1();
   return 0;
}
```

Output:

```
10 is end 4 is end 3
is end
```